

Oral Examination Sheet – Software Construction 2

HAW Hamburg · EMI · SO2 · Student: **Stiliani** · Examiner copy

1. Core OOP concepts, warm-up (5 min)

Ask short explanations with one Java example each:

- class vs. object
- encapsulation and why fields should usually be private
- inheritance vs. composition (is-a vs. has-a)
- method overloading vs. overriding

Expected / pass indicators: Can define class as blueprint, object as instance with state. Mentions information hiding, invariants, getters/setters or methods instead of public fields. Distinguishes extends from member fields. Overloading = same name, different parameter list, compile-time choice. Overriding = subclass provides its own implementation of inherited method. Can give small Java example.

2. True/False questionnaire: OOP and Java basics (5 min)

Student answers each statement with true or false. Remind her: **Please talk through your thoughts aloud while answering; reasoning is more important than guessing.**

Statement	True	False
In Java, a class can extend only one other class.		
A private field can be accessed directly from any other class in the same package.		
A constructor has the same name as the class and no return type.		
Method overloading means same method name with different parameter lists.		
A subclass can override an inherited method by providing a compatible method with the same signature.		
An interface can be implemented by multiple unrelated classes.		
A static field belongs separately to each object instance.		
For objects, == compares references, not logical content.		

Expected / pass indicators: Answer key:

1. True – Java has single class inheritance.
2. False – private means accessible only inside declaring class.
3. True – constructors have class name and no return type.
4. True – overload resolution uses parameter list.
5. True – same signature, compatible return type; @Override should compile.
6. True – interfaces define contracts for many implementations.
7. False – static belongs to class and is shared.
8. True – .equals() is for logical equality if implemented.

Bonus. Code reading: dynamic dispatch (if time remains)

Ask student to predict output and explain why. Then ask whether `Circle c = new Shape();` compiles.

```
1  class Shape {
2      public String name() { return "shape"; }
3      public void print() { System.out.println(name()); }
4  }
5
6  class Circle extends Shape {
7      @Override
8      public String name() { return "circle"; }
9  }
10
11 public class Demo {
12     public static void main(String[] args) {
13         Shape s = new Circle();
14         s.print();
15     }
16 }
```

 **Java**

Expected / pass indicators: Output: circle. Variable type is Shape, actual object is Circle; overridden methods are selected at runtime. print() is inherited, calls name(), and dynamic dispatch resolves to Circle.name(). `Circle c = new Shape();` does not compile without unsafe cast because not every Shape is a Circle.

Suggested passing evidence: student can explain OOP terms without memorized-only wording, justify true/false Java statements aloud, repair basic Java syntax/OOP errors, and write one correct loop with edge-case reasoning.

3. 2-minute writing task: small method

Student writes Java method `countPositive(int[] values)` that returns number of values greater than 0. Then explain null and empty-array behavior.

Expected / pass indicators: Acceptable solution:

```
1 public static int countPositive(int[] values) {
2     if (values == null) {
3         return 0;
4     }
5     int count = 0;
6     for (int value : values) {
7         if (value > 0) {
8             count++;
9         }
10    }
11    return count;
12 }
```

Java

Empty array returns 0. Null handling may also throw `IllegalArgumentException`; must be stated consistently.

4. Debugging task: find and repair errors (7 min)

Ask student to mark at least four problems and say correction.

```
1 public class BankAccount {
2     public double balance;
3
4     public void BankAccount(double balance) {
5         balance = balance;
6     }
7
8     public deposit(double amount) {
9         balance += amount;
10    }
11
12    public void withdraw(double amount) {
13        balance - amount;
14    }
15 }
```

Java

Expected / pass indicators: balance should be private for encapsulation. Constructor must have no return type: `public BankAccount(double balance)`. Field assignment needs `this.balance = balance;`. `deposit` missing return type: `public void deposit(double amount)`. `withdraw` expression not assigned: `balance -= amount;`. Good answer may also discuss validation: reject negative deposit/withdraw, prevent overdraft depending on requirements.

5. Closing transfer questions (3–5 min)

Choose two if time remains:

- Why use an interface instead of a concrete class?
- What does static mean for fields and methods?
- Why is `@Override` useful?
- What is difference between `==` and `.equals()` for objects?
- What should unit tests for `countPositive` cover?

Expected / pass indicators: Interface defines contract and enables multiple implementations. `static` belongs to class, not instance. `@Override` documents intent and catches signature mistakes. `==` compares references for objects; `.equals()` compares logical equality if implemented. Tests: null/empty, all negative, zero, positive mix, one element.